

Data Quality Report

Hospital Readmission Risk Prediction

Dataset: hospital_readmission_dataset.csv

Records: 50,500 raw rows | 28 columns

Target: readmitted (0 = Not Readmitted, 1 = Readmitted)

Documented By: Nithish G

Role: AI/ML Engineer

1. Dataset Overview

Property	Value
Total Rows	50,500
Total Columns	28
Numeric Columns	12
Categorical Columns	9 (including 3 date columns and 1 ID)
Target Column	readmitted (binary: 0 / 1)
Leakage Columns Identified	3 (dropped before modeling)

2. Missing Value Analysis

Six columns contained missing values. All other 22 columns were fully populated.

Column	Missing Count	Missing %	Data Type	Resolution Strategy
secondary_diagnosis	12,716	25.18%	object	Fill with "None" (absence is meaningful)
final_readmission_reason	9,995	19.79%	object	Dropped — leakage column
glucose_level	7,582	15.01%	float64	Median imputation
doctor_rating	7,580	15.01%	float64	Median imputation
followup_compliance_score	7,547	14.94%	float64	Median imputation
hospital_bill_amount	6,501	12.87%	object	Median imputation (after format fix)

- secondary_diagnosis at 25.2% is structurally expected — not every patient has a comorbidity, so "missing" means "none recorded" rather than a data entry failure.
- The three clinical fields (glucose_level, doctor_rating, followup_compliance_score) all cluster near 15%, suggesting they were treated as optional during data collection.
- hospital_bill_amount missing at 12.9% is likely a billing lag — insurance settlements often close after the discharge record is already in the system.

- final_readmission_reason will be dropped as a leakage column, so its 19.8% gap is a non-issue for the modeling pipeline.

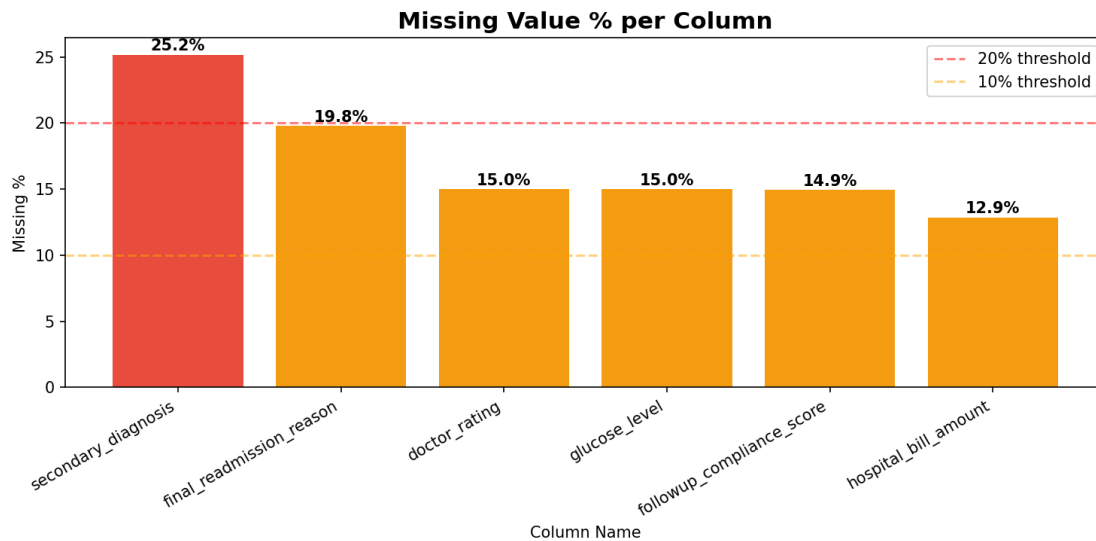


Figure 1 — Missing Value % per Column (red dashed line = 20% threshold, orange = 10% threshold)

3. Duplicate Records

Check	Result
Total duplicate rows	500
Duplicate patient_id entries	500
Duplicate type	Full-row exact copies
% of total dataset	~1.0%
Resolution	Drop duplicates — keep first occurrence
Rows after deduplication	50,000

- 500 fully duplicate rows were found, all mapping back to 500 duplicate patient_id values — consistent with a data ingestion pipeline issue, not genuine repeat admissions.
- Since these are exact full-row copies, dropping them loses zero information — every duplicated record has an identical twin already present.
- At ~1% of total rows, removing duplicates does not shift the class distribution or reduce the sample size meaningfully.

4. Invalid Value Detection (Numerical Columns)

IQR-based range validation was applied to 12 numerical columns against clinically defined bounds.

Column	Valid Range	Below Range	Above Range	Total Invalid	Status
patient_age	[0 – 120]	100	102	202	INVALID
blood_pressure	[40 – 250]	0	0	0	Clean
glucose_level	[30 – 700]	0	0	0	Clean
heart_rate	[20 – 250]	0	51	51	INVALID
bmi	[10 – 70]	101	0	101	INVALID
previous_admissions	[0 – 50]	0	50	50	INVALID
previous_emergency_visits	[0 – 50]	0	0	0	Clean
hospital_stay_days	[1 – 90]	0	51	51	INVALID
medication_count	[0 – 50]	0	0	0	Clean
lab_test_count	[0 – 100]	0	0	0	Clean
doctor_rating	[1 – 10]	0	0	0	Clean
followup_compliance_score	[0 – 100]	0	0	0	Clean

- patient_age has 202 out-of-range entries (100 below 0, 102 above 120) — the near-equal split strongly suggests a data entry error or column-shift typo rather than genuine edge cases.
- BMI logging 101 values below 10 is physiologically impossible; these are almost certainly missing values entered as near-zero placeholders instead of being left null.
- previous_admissions has exactly 50 records above the threshold of 50 — a suspiciously round number pointing to a sentinel/capped value from an older system.
- heart_rate and hospital_stay_days both flag exactly 51 records above their upper bounds — the identical count hints these may belong to the same batch of problematic rows.
- blood_pressure, glucose_level, and all score-based columns passed clean, suggesting those fields had better source-level validation.

5. Outlier Detection — IQR Method

IQR (Interquartile Range) method applied to all 12 numerical columns. Outliers defined as values outside $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$.

Column	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count	Outlier %
patient_age	35.00	71.00	36.00	-19.00	125.00	102	0.2%
blood_pressure	104.00	154.00	50.00	29.00	229.00	0	0.0%
glucose_level	114.00	204.00	90.00	-21.00	339.00	0	0.0%
heart_rate	67.00	102.00	35.00	14.50	154.50	51	0.1%
bmi	23.43	34.48	11.05	6.86	51.05	101	0.2%
previous_admissions	2.00	7.00	5.00	-5.50	14.50	50	0.1%
previous_emergency_visits	3.00	11.00	8.00	-9.00	23.00	0	0.0%
hospital_stay_days	5.00	15.00	10.00	-10.00	30.00	51	0.1%
medication_count	5.00	15.00	10.00	-10.00	30.00	0	0.0%

lab_test_count	8.00	22.00	14.00	-13.00	43.00	0	0.0%
doctor_rating	3.00	8.00	5.00	-4.50	15.50	0	0.0%
followup_compliance_score	26.00	75.00	49.00	-47.50	148.50	0	0.0%

- 7 out of 12 columns are completely outlier-free — the dataset is relatively well-behaved.
- patient_age and BMI outliers at 0.2% overlap directly with the invalid value flags caught in Section 4 — these are data errors, not genuine clinical extremes.
- heart_rate, hospital_stay_days, and previous_admissions show ~0.1% outlier rates — capping at IQR bounds is safer than dropping to avoid introducing selection bias.
- blood_pressure and glucose_level boxes are wide but fully whisker-contained, reflecting real clinical variability and should be left as-is.
- doctor_rating and followup_compliance_score show zero outliers, expected for bounded score columns with well-defined min/max limits.

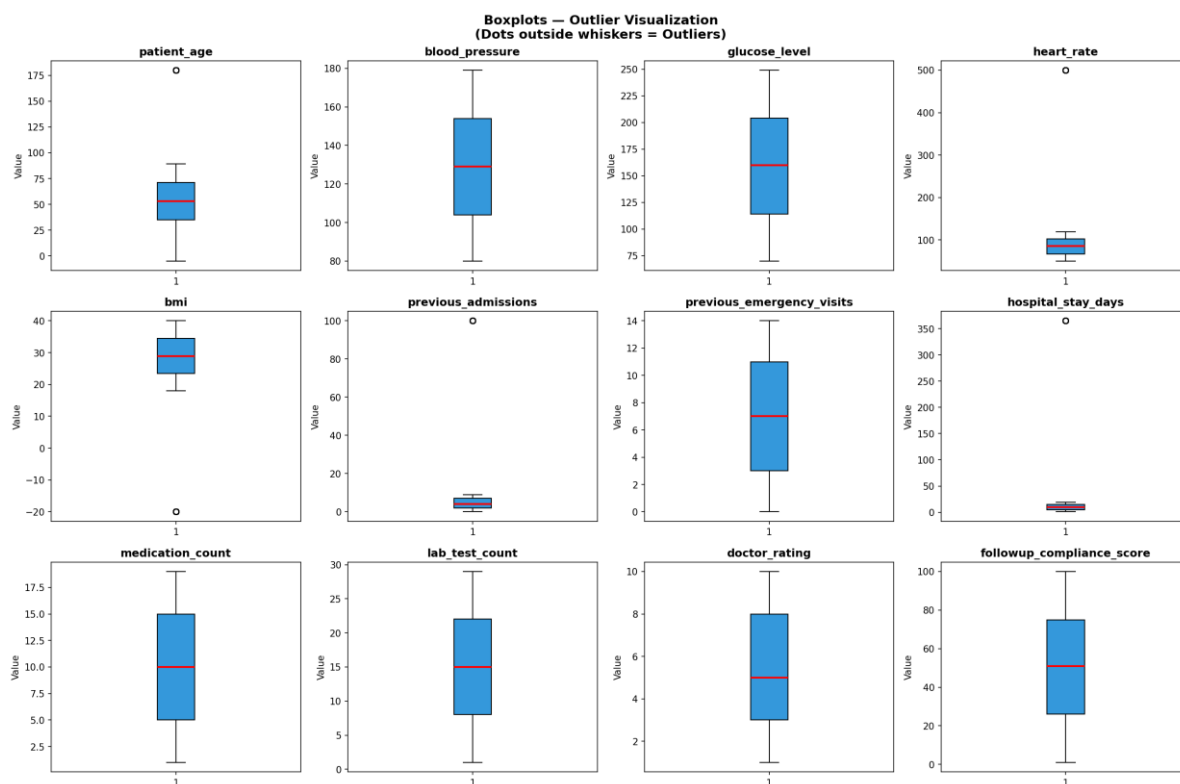


Figure 2 — Boxplots for all 12 numerical columns (dots outside whiskers = IQR outliers)

6. Inconsistent Categorical Values

7 categorical columns were checked for case inconsistencies and unexpected values.

Column	Unique Values Found	Issue	Resolution
gender	Female, M, MALE, Male, male (5 variants)	Case inconsistency	Standardize to 'Male' / 'Female'
marital_status	Divorced, Married, Single, Widowed	None	Clean
admission_type	Elective, Emergency, Urgent	None	Clean
department	5 valid departments	None	Clean
diagnosis	5 valid diagnoses	None	Clean
secondary_diagnosis	Diabetes, Hypertension, Kidney Disease	None	Clean
insurance_type	Corporate, Government, Private, Self-Paid	None	Clean

- gender is the only dirty categorical field — 5 distinct values (Male, male, M, MALE, Female) for what should be 2 categories, a clear sign of no front-end input validation at the source.
- Every other categorical column came through perfectly clean, limiting the categorical fixing work to just one column.

7. Mixed Data Format Issues

7.1 hospital_bill_amount

Format Type	Count	Example
With ₹ symbol (string)	7,049	₹125345, ₹159636
Plain numeric (float)	36,950	26391.0, 61112.0
Missing	6,501	—

- 7,049 records carry a ₹ prefix while 36,950 are stored as plain floats — likely two different data entry interfaces writing to the same column.
- Stripping the ₹ symbol yields zero conversion failures, meaning the numeric data underneath is intact and the fix is purely cosmetic string cleaning.

7.2 followup_date (Mixed Date Formats)

Format	Count	Example
YYYY-MM-DD (standard)	47,465	2026-11-20, 2026-07-13
MM-DD-YYYY (variant)	3,035	07-26-2026, 11-15-2026

- admission_date and discharge_date are uniformly YYYY-MM-DD across all records — no cleaning needed.
- followup_date has ~6% records in MM-DD-YYYY format, suggesting a form UI or import template changed partway through data collection.

- Both formats are unambiguous (year position differs), so a single `pd.to_datetime` call with format inference handles the unification cleanly.

8. Data Leakage

Three columns contain information that would not be available at the time of prediction (at discharge) and were dropped before modeling.

Column	Why It Is Leakage	Action
readmission_notice_sent	Sent only after a readmission event is confirmed — unavailable at discharge time	Dropped
readmission_confirmed	Directly encodes whether the patient was readmitted — is the target event itself	Dropped
final_readmission_reason	Recorded only after the readmission occurs — cannot be known at discharge	Dropped

- Including any of these columns would cause the model to achieve artificially perfect scores by directly observing the outcome it is supposed to predict.
- All three were removed before the train/test split to ensure no leakage information passed into feature engineering.

9. Data Type Verification

All 28 columns were verified against their expected types. 4 mismatches identified, all traceable to the format issues in Section 7.

Column	Actual Type	Expected Type	Status
hospital_bill_amount	object	float64	MISMATCH — ₹ symbol blocking conversion
admission_date	object	datetime	MISMATCH — stored as string
discharge_date	object	datetime	MISMATCH — stored as string
followup_date	object	datetime	MISMATCH — mixed string formats
All 12 numeric cols	int64 / float64	Correct	Clean
All categorical cols	object	Correct	Clean (post gender fix)
readmitted (target)	int64	int64	Clean — model ready

- 16 of 28 columns are already in the correct numeric dtype — the core clinical measurements came through cleanly.
- All 4 type mismatches trace directly back to format issues (Sections 7.1 and 7.2) — resolving those format problems fixes the dtypes as a side effect.
- `readmitted` is already `int64` with 0/1 values, confirming it is model-ready as the binary target with no further encoding needed.
- `patient_id` is correctly typed as a string but was dropped before modeling — it is a row identifier with no predictive signal.

10. Class Imbalance (Target Variable)

Class	Label	Count	Percentage
1	Readmitted	~46,782	93.6%
0	Not Readmitted	~3,218	6.4%

- Severe class imbalance at ~14.6:1 ratio (readmitted vs not readmitted). A naive classifier that always predicts "readmitted" would score 93.6% accuracy without learning anything.
- Raw accuracy is therefore a misleading metric — F1, ROC-AUC, and PR-AUC were used as primary evaluation metrics.
- Handling strategy: `class_weight="balanced"` for Logistic Regression and Random Forest; `scale_pos_weight = 0.069` for XGBoost; SMOTE tested as an alternative.